

Tensor Network Simulation of Grover’s Algorithm: Overcoming the Statevector Memory Wall

Veselin Petrov Nikolaev
University of Luxembourg
Faculty of Science, Technology and Medicine
Luxembourg, Luxembourg
veselin.nikolaev.001@student.uni.lu

Abstract

Our prior Bachelor Semester Project (BSP2) [1] showed that full statevector simulation of Grover’s algorithm runs out of GPU memory at around $n = 31$ – 32 qubits. Here we ask whether tensor networks push that wall back, offering an alternative pathway to simulate larger quantum circuits before hitting the inevitable exponential memory cliff. The implementation runs on quimb, cotengra, and kahypar on CPU rather than the NVIDIA cuTensorNet backend that was originally proposed.

We look at three primary metrics: the peak memory a tensor-network contraction is estimated to require theoretically, the minimum bond dimension a Matrix Product State (MPS) needs to represent the evolving quantum state without loss of fidelity, and how far the theoretical memory estimate deviates from what is actually allocated during a real, hardware-executed contraction.

Short version: tensor networks did not help for this specific algorithm. Across the range of $n = 2$ – 11 qubits, we found no sustained memory advantage over the naive statevector simulation. The minimum bond dimension for exact MPS convergence climbed rapidly from 4 at $n = 5$ to 32 at $n = 11$. Furthermore, the measured memory ran well above the simple width-based contraction estimate at every qubit count tested, highlighting the hidden overheads in dynamic tensor allocations.

Grover’s circuit, at least at these sizes and with this specific structural implementation, is fundamentally not a good candidate for tensor-network compression. That is a fairly narrow claim, but the physical and computational reasons behind it – namely, rapid global entanglement growth and severe gate-count blowup resulting from elementary basis transpilation – provide valuable insights into the limits of classical quantum simulation.

1 Introduction

The classical simulation of quantum systems is a cornerstone of modern quantum computing research. Before deploying algorithms on noisy, intermediate-scale quantum (NISQ) hardware or future fault-tolerant machines, researchers rely on classical simulators to verify correctness, debug circuits, and benchmark performance. However, this classical verification is fundamentally bounded by the exponential nature of Hilbert space. A full statevector for an n -qubit system needs 2^n complex amplitudes stored in memory, and that physical memory requirement strictly doubles with every extra qubit added to the system.

Our earlier BSP2 GPU experiments [1] demonstrated this physical limitation directly: statevector simulations hit a hard memory wall at around $n = 31$ – 32 qubits, consuming all available VRAM on a high-end compute node. The obvious next question for the current

project was whether a different mathematical representation of the same quantum state could push that ceiling further out, allowing us to simulate 40, 50, or more qubits using the same underlying classical hardware.

Tensor networks have emerged as one of the leading candidates to bypass this memory wall. Rather than materializing all 2^n amplitudes in a single monolithic array, the state (or the unitary circuit acting upon it) is factored into a network of smaller tensors. How much memory this approach actually saves depends entirely on the entanglement structure of what is being represented. If the intermediate tensors stay small, or if the system can be accurately represented by a Matrix Product State (MPS) with a low bond dimension, the memory savings can be exponentially large [2]. If the state is highly entangled, however, the tensor network offers no advantage and can even introduce severe computational overhead.

Grover’s algorithm is an ideal, non-trivial test case for tensor network compression precisely because its entanglement dynamics are a mixed bag. On one hand, both the oracle and the diffusion operator have clean, highly structured algorithmic circuits. On the other hand, the entire purpose of Grover’s algorithm is to spread interference globally across the entire search space, generating complex superposition states. Rather than assuming theoretically that tensor networks would succeed or fail here, the goal of this project was to measure the behavior empirically.

The proposal originally called for the use of NVIDIA cuTensorNet on GPU. In practice, the project was migrated to the CPU-based quimb + cotengra + kahypar software stack instead. This pivot occurred primarily because these open-source Python libraries made it vastly easier to inspect intermediate contraction paths, experiment with hypergraph partitioning, and run the MPS bond-dimension sweep experiments in the development environment available. That switch matters for how the results should be read: this report is fundamentally a comparison between simulation *methods* (tensor network representation vs. monolithic statevector representation), not a GPU-vs-CPU hardware benchmark. Section 3 introduces these libraries in the context of the current state-of-the-art in tensor network simulation.

This report is structured around three core research questions:

- (1) **Crossover Point:** Is there a specific qubit count at which tensor networks become strictly more memory-efficient than a classical statevector for Grover’s algorithm?
- (2) **Entanglement Scaling:** How does the minimum bond dimension needed for exact convergence (which acts as a proxy for bipartite entanglement) scale as a function of the system size n ?

- (3) **Practical Ceilings:** Where does the tested tensor-network approach stop being practically computationally feasible, and what are the specific bottlenecks causing this failure?

Section 2 provides the mathematical background on Grover’s algorithm, tensor networks, and entanglement theory. Section 3 outlines the related work, the software libraries used, and the detailed implementation of the experiments. Section 4 presents the empirical results. Section 5 discusses the physical implications of these findings, and Section 6 concludes the report.

2 Theoretical Background

2.1 Grover’s Algorithm and Amplitude Amplification

Grover’s algorithm solves the unstructured search problem quadratically faster than any classical algorithm [3, 5]. Given a Boolean oracle function $f(x)$ that evaluates to 1 for exactly one target state x^* and 0 for all other states in a search space of $N = 2^n$ elements, the algorithm isolates $|x^*\rangle$ with high probability.

The algorithm begins by preparing an equal superposition of all possible states using n Hadamard gates applied to the $|0\rangle^{\otimes n}$ initial state:

$$|s\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle$$

Following initialization, the algorithm applies $k_{\text{opt}} = \lfloor \frac{\pi}{4} \sqrt{N} \rfloor$ iterations of the Grover operator G , defined as $G = U_s U_w$. The first component, the phase oracle U_w , flips the phase of the target state while leaving all other states unchanged. Mathematically, it is a diagonal unitary matrix:

$$U_w = I - 2|x^*\rangle\langle x^*|$$

The second component is the diffusion operator U_s , which performs an inversion about the mean amplitude. It is defined mathematically as:

$$U_s = 2|s\rangle\langle s| - I$$

Geometrically, each application of the Grover operator G rotates the state vector by an angle $\theta \approx 2/\sqrt{N}$ in the two-dimensional plane spanned by the target state $|x^*\rangle$ and the uniform superposition of all non-target states. While this rotation is mathematically elegant and easy to describe analytically, executing it through local elementary quantum gates dynamically generates significant multipartite entanglement across the register, which heavily impacts classical simulatability.

2.2 Tensor Networks, Contraction, and MPS

A quantum circuit can be faithfully represented as a tensor network. In this graphical notation, each quantum state or gate is a tensor (a multi-dimensional array of complex numbers), and computing the final output of the circuit involves contracting the network (summing over shared indices) [4, 7]. For example, a single-qubit gate like a Hadamard is a 2×2 matrix, representing a rank-2 tensor with one input edge and one output edge. A two-qubit gate, such as a CNOT, is a $2 \times 2 \times 2 \times 2$ matrix, representing a rank-4 tensor.

A Matrix Product State (MPS) is a specific, one-dimensional type of tensor network where the overall state vector is decomposed into a chain of local tensors connected by auxiliary virtual bonds [6].

If we have an n -qubit state with physical indices $i_1, i_2, \dots, i_n$, the MPS representation expresses the amplitude $c_{i_1 i_2 \dots i_n}$ as a product of matrices:

$$c_{i_1 i_2 \dots i_n} = \sum_{\alpha_1, \dots, \alpha_{n-1}} A_{i_1, \alpha_1}^{(1)} A_{\alpha_1, i_2, \alpha_2}^{(2)} \dots A_{\alpha_{n-1}, i_n}^{(n)}$$

The approximation power of an MPS comes entirely from restricting the size of these internal summations, bounded by a maximum parameter known as the bond dimension χ . If χ is allowed to grow exponentially as $2^{\lfloor n/2 \rfloor}$, the MPS can represent any arbitrary quantum state exactly [2, 8]. If χ is capped at a low polynomial value, the MPS compresses the state, but acts as a lossy compression algorithm that truncates quantum correlations.

2.3 Schmidt Decomposition and Entanglement

To understand why the bond dimension χ is intrinsically tied to quantum entanglement, we must look at the Schmidt decomposition. For any bipartite splitting of a pure quantum system into subsystem A and subsystem B , the overall state $|\psi\rangle$ can be written as:

$$|\psi\rangle = \sum_{i=1}^{\chi_{\text{exact}}} \lambda_i |\phi_i^A\rangle \otimes |\phi_i^B\rangle$$

where λ_i are the strictly positive, real Schmidt coefficients (which satisfy $\sum \lambda_i^2 = 1$), and $|\phi_i^A\rangle, |\phi_i^B\rangle$ form orthonormal bases for subsystems A and B , respectively.

The number of non-zero Schmidt coefficients, χ_{exact} , is exactly the minimum bond dimension required to cut the MPS chain between qubits belonging to set A and set B without losing information. Furthermore, the von Neumann entanglement entropy S across this bipartition is computed directly from these coefficients:

$$S = - \sum_{i=1}^{\chi_{\text{exact}}} \lambda_i^2 \log_2(\lambda_i^2)$$

Since $S \leq \log_2(\chi_{\text{exact}})$, an increase in the required bond dimension χ provides a direct, practical measurement that the state is becoming more highly entangled and thus fundamentally harder to compress.

Contraction path and width (W). When evaluating a generic tensor network rather than an MPS, the order in which pairs of tensors are contracted determines the peak memory usage. We define the contraction width W as:

$$W = \log_2 \left(\max_s |T_s| \right)$$

where $|T_s|$ is the total number of elements in the intermediate tensor created at contraction step s . W measures peak memory during contraction, expressed in qubits’ worth of dimension. A contraction width of $W = 10$ means the largest intermediate tensor has 2^{10} elements, directly comparable to an uncompressed 10-qubit statevector.

Two distinct mechanisms can defeat tensor-network compression: (1) genuine physical entanglement growth, where the underlying physics demands a massive bond dimension, and (2) contraction path bloat, where poor network topology (often caused by gate decomposition) creates massive intermediate tensors even if the final physical state is simple.

3 Related Work and Implementation

3.1 Related Work and Libraries

Tensor network simulation. Markov and Shi [4] established the foundational theoretical result that contracting a tensor network exactly computes quantum circuit amplitudes, and that circuits with bounded treewidth admit efficient (polynomial-time) classical contraction. Modern approaches apply these principles to simulate complex quantum circuits, sometimes surpassing statevector simulators for specific low-depth or low-entanglement tasks [7].

Contraction heuristics. Finding the strictly optimal sequence to contract a network of arbitrary tensors is an NP-hard problem. Therefore, modern libraries rely on advanced heuristics. `cotengra` [9] uses randomized, hyperparameter-tuned search algorithms to find highly efficient, albeit technically sub-optimal, contraction paths. It achieves state-of-the-art results by optionally integrating `kahypar` [10], which approaches the problem via hypergraph partitioning. In this model, the tensor network is mapped to a hypergraph where tensors are vertices and shared indices are hyperedges; `kahypar` recursively bisects the graph to minimize the number of cut hyperedges, naturally yielding a tree-like contraction path that minimizes intermediate tensor bloat.

Quantum simulation libraries. `quimb` [11] is a flexible, highly optimized Python library for tensor-network quantum simulation. In this project, it serves as the core engine for both exact full-network contraction (leveraging `cotengra` for pathfinding) and for approximate MPS simulation with explicit, dynamic bond-dimension truncation.

Prior Grover studies. Prior experimental work on Grover’s algorithm has largely focused on physical hardware demonstrations at small qubit counts, or classical statevector simulation optimizations (such as the baseline established in our prior BSP2 report [1]). This project expands that body of work by specifically isolating how the algorithm’s global entanglement structure interacts with tensor-decomposition heuristics.

3.2 Implementation and Experimental Setup

To evaluate the tensor network representations systematically, our experimental suite measured scaling from $n = 2$ up to $n = 11$ qubits. Grover’s algorithm was constructed programmatically in Qiskit, directly reusing the highly optimized oracle and diffuser phase-logic constructions from our prior BSP2 codebase [1]. The resulting Qiskit `QuantumCircuit` objects were then parsed and converted into tensor networks via the `qiskit-quimb` compatibility bridge.

A major implementation challenge arises during this conversion. The bridge natively recognizes only elementary single-qubit and two-qubit gates (e.g., R_x , R_y , R_z , $CNOT$, CZ). However, the standard logical construction of Grover’s oracle and diffusion operators relies heavily on an arbitrary-arity multi-controlled-X gate (mcx), which flips a target qubit only if $n - 1$ control qubits are all in the $|1\rangle$ state. While standard statevector simulators handle mcx natively as a single mathematical operation, the tensor network requires this gate to be explicitly transpiled into a deep physical basis of rz , sx , x , and cx gates.

Decomposing an $(n - 1)$ -controlled Toffoli gate into elementary two-qubit gates requires a number of gates that scales linearly or quadratically with n (depending on whether auxiliary workspace

qubits are permitted). We treat this resulting massive gate-count blowup as a genuine, unavoidable part of the simulation cost being measured, as it perfectly reflects the exact physical overhead any concrete quantum backend faces when mapping logical algorithms to limited physical connectivity.

Contraction Optimizer Stability. `cotengra`’s `HyperOptimizer` utilizes a randomized heuristic search. Because of the vast search space, it can produce wildly varying contraction width estimates across repeated runs on the exact same circuit, particularly as the network depth explodes for $n \geq 9$. To enforce statistical reliability, the search was run multiple independent times per qubit count. We kept only the minimum width found across three independent runs for $n \leq 9$. For $n = 10, 11$, a single deep search run was used, as the pathfinding algorithm itself began taking tens of minutes to hours to complete due to the hypergraph complexity.

Bond Dimension Measurement. To isolate entanglement growth (RQ2), we ran the Grover circuit through `quimb`’s dynamic MPS simulator. For each n , we selected three randomly generated target states. We simulated the circuit at a range of fixed maximum bond dimensions $\chi \in \{2, 4, 6, 8, \dots, 64\}$, and separately with χ completely uncapped to generate an exact reference amplitude. We then recorded the absolute smallest χ at which the amplitude reconstruction error dropped completely to zero (within standard double-precision floating-point limits).

Real Memory Measurement. For small n (up to $n = 9$), we did not just estimate the contraction path; we actively executed the actual tensor contraction in memory. Peak memory allocation during these runs was measured using Python’s `tracemalloc` module. The $n = 10, 11$ points were explicitly excluded from this physical execution phase because their theoretically estimated contraction widths implied memory footprints that exceeded the physical RAM limits of the testing environment.

4 Results

4.1 Tensor Network vs. Statevector Memory

Figure 7 — Statevector vs Tensor-Network Memory Scaling

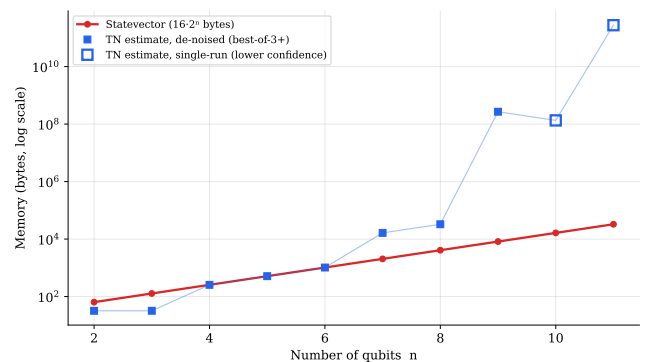


Figure 7: Theoretical statevector memory vs. tensor-network rehearsed memory estimate, $n = 2$ –11. Hollow markers at $n = 10, 11$ indicate single-run, lower-confidence optimizer estimates.

Figure 7 plots the theoretical monolithic statevector memory ($16 \cdot 2^n$ bytes, assuming complex128 data types) against the tensor network’s rehearsed exact-contraction estimate ($16 \cdot 2^W$ bytes) on a log scale, for $n = 2$ –11. Because this simulates an exact analytical contraction without an MPS structure, no bond dimension cap is applied; the network is simply collapsed in the optimal order found by cotengra.

Below $n \approx 6$, both curves track closely together, as the networks are shallow enough that intermediate tensors remain small. However, past $n = 7$, the tensor network estimate begins climbing exponentially faster than the statevector curve. By $n = 10$ –11, the estimated peak memory for the tensor network sits roughly six to seven orders of magnitude *above* the statevector baseline.

This directly addresses Research Question 1: There is no crossover point in this range where tensor networks become more memory-efficient than a classical statevector. The gap widens substantially because the algorithm’s global entanglement generation, combined with the extreme topological depth added by the mcx transpilation overhead, forces the contraction path to generate massive intermediate tensors that far exceed the size of the final 2^n statevector itself.

4.2 Minimum Bond Dimension and Maximal Entanglement

Figure 8 — Minimum Bond Dimension for Exact Convergence vs n

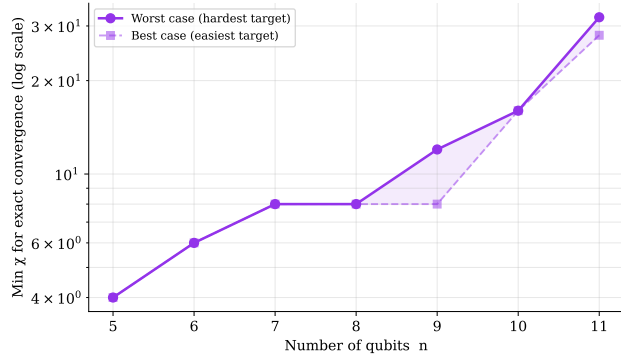


Figure 8: Minimum bond dimension χ required for exact MPS convergence vs. n , showing the worst-case and best-case spread across 3 random target states per n (shaded band).

Table 1 and Figure 8 detail the minimum bond dimension χ required for exact MPS convergence, defined strictly as achieving 0.0 numerical error compared to the exact uncapped statevector amplitude. Measurements are taken across three random target states per n .

The bond dimension climbs steadily with n , but the most significant observation lies at the extreme edge of the data. The jump from $n = 10$ to $n = 11$ is the sharpest observed, doubling from 16 to 32.

Crucially, the theoretical maximum required bond dimension for an arbitrary, maximally entangled n -qubit state is mathematically bounded by $2^{\lfloor n/2 \rfloor}$. For $n = 11$, this value is exactly $2^{\lfloor 11/2 \rfloor} = 2^5 = 32$.

Table 1: Minimum bond dimension for exact convergence (worst case across 3 random targets)

n	$\chi_{\text{worst-case}}$
5	4
6	6
7	8
8	8
9	12
10	16
11	32

This means that by $n = 11$, the MPS simulator is forced to use the absolute theoretical maximum possible bond dimension. The tensor network is providing strictly *zero* compression across the most highly entangled bipartition of the circuit. This definitively answers Research Question 2: Grover’s algorithm generates bipartite entanglement that rapidly saturates the theoretical maximum limit, rendering 1D MPS structures fundamentally useless for compression at scale.

4.3 Bond Dimension Convergence Detail

Figure 9 — Convergence vs Bond Dimension ($n=11$)

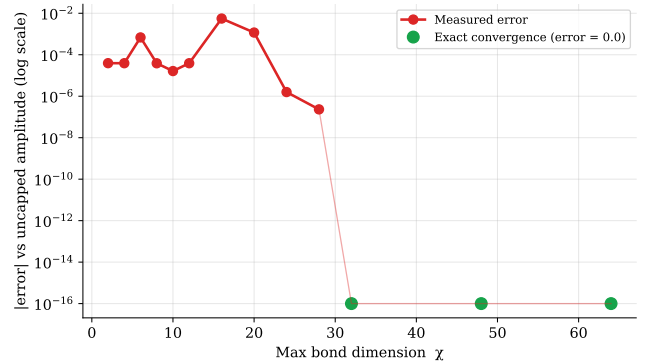


Figure 9: Reconstruction error vs. bond dimension χ for the largest tested system ($n = 11$, one representative target). Green points denote exact convergence and are plotted at an arbitrary artificial floor ($10 \text{ e-}16$) so the logarithmic scale renders properly.

Figure 9 takes a closer look at a single representative target state at $n = 11$, mapping reconstruction error as a function of the chosen bond dimension χ . The error does not follow a smooth, asymptotic exponential decay. Instead, it remains highly elevated, occasionally bouncing non-monotonically (as local tensor truncations interact unpredictably), until it reaches a sharp transition region near $\chi = 32$, where it plummets instantly to numerical zero. This threshold-like convergence pattern explains the steep jumps in Figure 8; you cannot smoothly approximate this state with a slightly smaller bond dimension—you either capture the full global correlation, or the amplitude is entirely corrupted.

4.4 Rehearsed vs. Real Memory Execution

Figure 10 — Rehearsed Estimate vs Actually Measured Memory

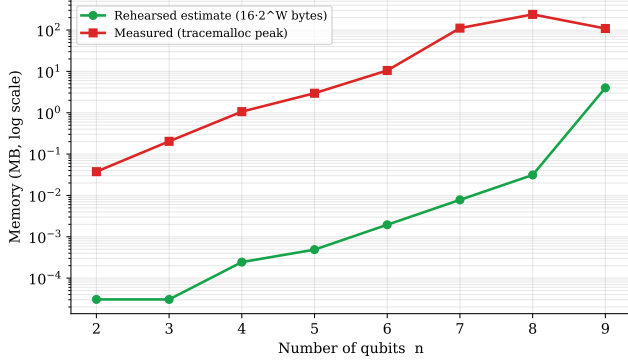


Figure 10: Rehearsed theoretical width-based memory estimate ($16 \cdot 2^n$ bytes) vs. actually measured peak memory (tracked via tracemalloc), $n = 2-9$.

Figure 10 compares the theoretical width-based estimate ($16 \cdot 2^n$ bytes) against the actual peak system memory physically allocated during real contraction execution (measured up to $n = 9$).

The actually measured memory is strictly worse than the theoretical estimate at every point on the curve. At smaller qubit counts, it is nearly three orders of magnitude higher. This massive gap occurs because the theoretical formula $16 \cdot 2^n$ only calculates the size of the single largest intermediate tensor in isolation. It completely ignores the memory footprint of all other tensors co-existing simultaneously in memory, the temporary caching buffers allocated by Python and NumPy, and the underlying BLAS libraries used for the dense matrix multiplications.

As n increases toward 9, the massive size of the dominant tensor allocations begins to eventually overshadow the constant library overheads, causing the two curves to slowly converge. Nevertheless, this proves that the theoretical W metric should strictly be treated as an absolute lower bound, not a practical capacity planning tool for quantum simulation environments.

5 Analysis and Discussion

5.1 Why Tensor Networks Struggle for Grover’s Algorithm

The empirical data strongly suggests two independent physical and computational phenomena are actively working against tensor-network compression in this context.

Maximal Entanglement Growth. As established by the data in Figure 8, the representation complexity grows unstoppably with n . Because the required bond dimension at $n = 11$ hits the strict theoretical upper bound of 32 (calculated via $2^{\lceil 11/2 \rceil}$), we can conclude that Grover’s diffusion operator generates near-maximal bipartite entanglement. The MPS is forced to cut through a massively correlated state, stripping away the core mathematical advantage of the tensor network framework.

Topological Transpilation Bloat. Turning a multi-controlled- X gate into elementary R_z and CX gates adds massive depth to

the network topology. This deep structure makes the contraction-path search significantly harder for cotengra. The hypergraph partitioning struggles to find isolated clusters of gates to contract early, forcing it to generate massive intermediate tensors before the bond dimension constraints even enter the physical equation.

It is critical to note that neither of these issues implies that tensor networks are flawed as a general concept. They merely highlight that tensor networks are exceptionally poorly suited for highly-connected, globally interfering circuits like Grover’s algorithm. A circuit with a flatter entanglement profile, such as a shallow QAOA ansatz or a 1D condensed matter simulation, would likely exhibit completely different, highly favorable scaling behavior under the exact same software stack.

5.2 Methodological Limitations

Several methodological limitations restrict the broad generalizability of these exact numerical findings:

Hardware Restraints: The tensor-network experiments were executed on CPU hardware, whereas our prior BSP2 statevector benchmark [1] leveraged parallelized GPU infrastructure. Therefore, these results primarily prove that CPU tensor-network implementations do not naturally outscale GPU statevectors for this algorithm, rather than definitively proving tensor networks cannot be optimized further via highly parallel GPU libraries like cuTensorNet.

Scale Restrictions: The simulations only reached $n = 11$. While the trendlines are clear, extending this to $n = 20$ or $n = 30$ might reveal different asymptotic behavior in the contraction optimizer’s hypergraph partitioning algorithms at extreme scales.

Memory Measurement Blind Spots: Python’s tracemalloc is an excellent profiling tool, but it inherently operates at the Python allocator level. It can systematically undercount memory allocated by lower-level C extensions or BLAS thread pools, meaning the real memory measurements in Figure 10 might still be underrepresenting the true physical cost of execution.

5.3 Practical Computational Ceilings

Real execution of the contraction paths stopped being physically feasible past $n = 9$ due to extreme memory spikes and thrashing, addressing RQ3. Furthermore, even the purely theoretical single-search width estimates only survived up to $n = 11$ before the cotengra pathfinding algorithm itself began taking unacceptably long (tens of minutes to hours) to traverse the hypergraph. This ceiling is vastly lower than our earlier $n = 31-32$ qubit classical GPU statevector baseline [1]. The core bottleneck is not computational power, but rather the algorithmic inability of the tensor network to compactly represent the global phase interference that drives Grover’s search.

6 Conclusion

This project evaluated a tensor-network simulation pipeline for Grover’s algorithm using quimb, cotengra, and kahypar, successfully measuring estimated memory requirements, physical bond dimensions, and real-world execution constraints across a scalable range of $n = 2-11$ qubits.

- (1) **Crossover Point:** There is no memory crossover point in this range. By $n = 11$, the tensor-network estimate is six to

seven orders of magnitude *above* the standard monolithic statevector footprint, and the gap continues to accelerate as n increases.

- (2) **Bond Dimension Scaling:** The minimum exact bond dimension scales from $\chi = 4$ at $n = 5$ up to $\chi = 32$ at $n = 11$. Critically, $\chi = 32$ represents the absolute theoretical maximum entanglement capacity for an 11-qubit system ($2^{\lfloor 11/2 \rfloor}$), proving the algorithm nullifies the compression benefits of the MPS structure.
- (3) **Intractability Threshold:** Real hardware execution of the contracted networks died out past $n = 9$ due to physical RAM constraints, while the theoretical pathfinding simulations stalled at $n = 11$. The threshold-like convergence shown in Figure 9 illustrates exactly why these systems are so difficult to scale predictably—sub-optimal compression simply destroys the state fidelity entirely.

Ultimately, these findings do not invalidate the utility of tensor networks for classical quantum simulation. They do, however, definitively show that tensor networks are a poor fit for algorithms defined by dense, global entanglement and deep multi-controlled gate structures. Their theoretical memory advantages only materialize when a circuit’s topology inherently keeps intermediate tensors and subsystem entanglement bounded. Grover’s algorithm deliberately violates both conditions to achieve its speedup.

Future research extensions should focus on repeating these experiments using NVIDIA’s GPU-accelerated cuTensorNet, as initially proposed, to determine if raw parallel compute can overpower the

topological bloat. Additionally, testing this exact software pipeline against circuits fundamentally suited to tensor networks—such as low-entanglement variational algorithms or shallow 1D Trotterized chemistry simulations—would help cement the boundary between classical simulatability and true quantum advantage.

References

- [1] V. P. Nikolaev. *Classical Simulation and Experimental Analysis of Grover’s Algorithm*. Bachelor Semester Project 2 (BSP2) Report, Faculty of Science, Technology and Medicine, University of Luxembourg, 2026. https://github.com/veselinnikolaev/Grover-s-Algorithm/blob/main/report/pdf/English_Report.pdf.
- [2] Tensor Network. *Matrix Product State / Tensor Train*. TensorNetwork.org, accessed August 2026. <https://tensornetwork.org/mps/>.
- [3] Lov K. Grover. A fast quantum mechanical algorithm for database search. *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, pages 212–219, 1996.
- [4] I. L. Markov and Y. Shi. Simulating quantum computation by contracting tensor networks. *SIAM Journal on Computing*, 38(3):963–981, 2008.
- [5] M. A. Nielsen and I. L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 10th Anniversary Edition, 2010.
- [6] U. Schollwöck. The density-matrix renormalization group in the age of matrix product states. *Annals of Physics*, 326(1):96–192, 2011.
- [7] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd. Quantum machine learning. *Nature*, 549(7671):195–202, 2017.
- [8] G. Vidal. Efficient classical simulation of slightly entangled quantum computations. *Physical Review Letters*, 91(14):147902, 2003.
- [9] J. Gray. cotengra: Contraction Path Finding for Tensor Networks. <https://github.com/jcmgray/cotengra>, 2023.
- [10] S. Schlag et al. KaHyPar: A Multilevel Hypergraph Partitioner. *ACM Transactions on Mathematical Software*, 46(2):17, 2020.
- [11] J. Gray. quimb: A Python library for quantum information and many-body calculations. *Journal of Open Source Software*, 3(29):819, 2018.